Agtron happens to be walking past (his timing is impeccable). He looks at what you're doing, smiles, and asks if he can show you something. He shows you what's happening while your program is sorting all those competitors by adding a `setInterval` intended to log to the console once per second:

```
start = new Date()
setInterval ->
  console.log "Clock tick after #{(new Date()-start)/1000} seconds"
, 1000
```

You expect to see a *clock tick* each second, but that's *not* what happens:

```
Loading and processing data
Starting server on port 8888
Data loaded in 4.636 seconds
Clock tick at 4.643 seconds
Clock tick at 4.643 seconds
```

The program does only *one thing at a time*. If it spends 5 seconds sorting an array, then nothing else happens during those 5 seconds—the event loop experiences a blackout. Then, once the event loop is free, you see two events that have been waiting to be handled occur immediately one after the other.

9.2.1. Events and blackouts

When something happens that the event loop needs to handle, then the event loop receives an event. For example, when you evaluated the `setInterval` call with a function and a delay of 1,000 milliseconds, you were requesting that an event occur on the event loop in 1,000 milliseconds and for the function you supplied to be invoked as the handler for the event:

```
setInterval ->
  console.log "Clock tick after #{(new Date()-start)/1000} seconds"
, 1000
```

Similarly, when you request a file to be read asynchronously, you're asking the file to be read outside of the event loop and for an event to be triggered